

Практическое занятие 32: Деплой проекта

Цель занятия:

- Научиться развёртывать Django-проект на сервере в продакшн-среде.
 - Понять общие принципы и этапы деплоя приложения.
 - Настроить необходимые компоненты для работы приложения: веб-сервер, приложение WSGI, база данных.
 - Обеспечить безопасность и стабильность работы приложения в продакшн-среде.
-

Задачи занятия

1. Общие шаги деплоя Django-проекта
 2. Настройка сервера для деплоя
 3. Установка необходимых зависимостей на сервере
 4. Клонирование проекта и настройка виртуального окружения
 5. Настройка базы данных
 6. Настройка Gunicorn как WSGI-сервера
 7. Настройка веб-сервера Nginx для обработки запросов
 8. Запуск и проверка работы приложения
-

Задача 1: Общие шаги деплоя Django-проекта

1.1. Этапы деплоя

- **Настройка сервера:** Установка операционной системы и необходимых пакетов.
- **Установка зависимостей:** Установка Python, виртуального окружения и необходимых библиотек.
- **Клонирование проекта:** Получение кода приложения на сервере.
- **Настройка базы данных:** Установка и настройка СУБД, создание базы данных.
- **Настройка приложения:** Применение миграций, сборка статических файлов.
- **Настройка WSGI-сервера:** Установка и настройка Gunicorn или аналогичного сервера.
- **Настройка веб-сервера:** Установка и настройка Nginx для обработки HTTP-запросов.
- **Настройка безопасности:** Обеспечение безопасности приложения и сервера.

- **Тестирование:** Проверка работы приложения в продакшн-среде.
-

Задача 2: Настройка сервера для деплоя

2.1. Подготовка сервера

Шаги:

1. Выбор операционной системы

- Рекомендуется использовать серверную версию Linux, например, Ubuntu Server или CentOS.
- Убедитесь, что у вас есть доступ к серверу с правами администратора (root) или через `sudo`.

2. Обновление пакетов системы

```
sudo apt update
sudo apt upgrade
```

3. Установка необходимых пакетов

Установите необходимые инструменты и библиотеки:

```
sudo apt install python3-pip python3-dev python3-venv nginx
```

Пояснение:

- `python3-pip` : Менеджер пакетов pip для Python 3.
 - `python3-dev` : Заголовочные файлы для разработки на Python.
 - `python3-venv` : Инструмент для создания виртуальных окружений.
 - `nginx` : Веб-сервер для обработки HTTP-запросов.
-

Задача 3: Установка необходимых зависимостей на сервере

3.1. Установка Python и виртуального окружения

Шаги:

1. Проверка установленной версии Python

```
python3 --version
```

2. Создание пользователя для приложения (опционально)

Для безопасности рекомендуется создать отдельного пользователя для запуска приложения:

```
sudo adduser django_user
```

3. Авторизация под созданным пользователем

```
sudo su - django_user
```

3.2. Установка Git (если требуется)

Шаги:

1. Установка Git

```
sudo apt install git
```

Задача 4: Клонирование проекта и настройка виртуального окружения

4.1. Клонирование проекта из репозитория

Шаги:

1. Переход в домашний каталог пользователя

```
cd ~
```

2. Клонирование репозитория

```
git clone https://your-repo-url.git
```

Примечание: Замените `https://your-repo-url.git` на URL вашего репозитория.

4.2. Настройка виртуального окружения

Шаги:

1. Переход в директорию проекта

```
cd your_project_directory
```

2. Создание виртуального окружения

```
python3 -m venv venv
```

3. Активация виртуального окружения

```
source venv/bin/activate
```

4. Установка зависимостей из requirements.txt

```
pip install --upgrade pip  
pip install -r requirements.txt
```

Задача 5: Настройка базы данных

5.1. Установка и настройка PostgreSQL (или другой СУБД)

Шаги:

1. Установка PostgreSQL

```
sudo apt install postgresql postgresql-contrib
```

2. Создание базы данных и пользователя

```
sudo -u postgres psql
```

В интерактивной оболочке PostgreSQL выполните:

```
CREATE DATABASE your_db_name;  
CREATE USER your_db_user WITH PASSWORD 'your_db_password';  
ALTER ROLE your_db_user SET client_encoding TO 'utf8';  
ALTER ROLE your_db_user SET default_transaction_isolation TO 'read  
committed';
```

```
ALTER ROLE your_db_user SET timezone TO 'UTC';  
GRANT ALL PRIVILEGES ON DATABASE your_db_name TO your_db_user;  
\q
```

Пояснение:

- `your_db_name` : Название вашей базы данных.
- `your_db_user` : Имя пользователя базы данных.
- `your_db_password` : Пароль для пользователя базы данных.

5.2. Настройка подключения к базе данных в продакшн-настройках

Шаги:

1. Откройте файл `production_settings.py`
2. Обновите настройки базы данных:

```
# production_settings.py  
  
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql_psycopg2',  
        'NAME': 'your_db_name',  
        'USER': 'your_db_user',  
        'PASSWORD': 'your_db_password',  
        'HOST': 'localhost',  
        'PORT': '',  
    }  
}
```

5.3. Применение миграций и сборка статических файлов

Шаги:

1. Применение миграций

```
python manage.py migrate --settings=your_project.production_settings
```

2. Сборка статических файлов

```
python manage.py collectstatic --  
settings=your_project.production_settings
```

Примечание: При выполнении этой команды вам будет предложено подтвердить действие. Введите `yes`.

Задача 6: Настройка Gunicorn как WSGI-сервера

6.1. Установка Gunicorn

Шаги:

1. Установка Gunicorn в виртуальном окружении

```
pip install gunicorn
```

2. Добавьте Gunicorn в `requirements.txt`

```
echo 'gunicorn' >> requirements.txt
```

6.2. Тестовый запуск приложения с Gunicorn

Шаги:

1. Запуск Gunicorn

```
gunicorn --bind 0.0.0.0:8000 your_project.wsgi:application --  
settings=your_project.production_settings
```

Пояснение:

- `your_project.wsgi:application` : Путь к вашему WSGI-приложению.
- `--settings=your_project.production_settings` : Указание настроек продакшн.

2. Проверьте, что приложение работает по адресу `http://your_server_ip:8000`.

6.3. Настройка системного сервиса для Gunicorn

Шаги:

1. Создайте файл службы для `systemd`

```
sudo nano /etc/systemd/system/gunicorn.service
```

2. Добавьте следующее содержимое в файл `gunicorn.service` :

```
[Unit]
Description=gunicorn daemon
After=network.target

[Service]
User=django_user
Group=www-data
WorkingDirectory=/home/django_user/your_project_directory
ExecStart=/home/django_user/your_project_directory/venv/bin/gunicorn --
access-logfile - --workers 3 --bind
unix:/home/django_user/your_project_directory/your_project.sock
your_project.wsgi:application --
settings=your_project.production_settings

[Install]
WantedBy=multi-user.target
```

****Пояснение:****

- ****`User`****: Пользователь, под которым будет запускаться Gunicorn.
- ****`WorkingDirectory`****: Путь к директории вашего проекта.
- ****`ExecStart`****: Команда запуска Gunicorn с указанием сокета Unix.

3. ****Сохраните файл и выйдите из редактора.****

4. ****Запустите и активируйте службу Gunicorn****

```
```bash
sudo systemctl start gunicorn
sudo systemctl enable gunicorn
```

5. **Проверьте статус службы**

```
sudo systemctl status gunicorn
```

---

## Задача 7: Настройка веб-сервера Nginx для обработки запросов

### 7.1. Настройка конфигурации Nginx

## Шаги:

### 1. Создайте файл конфигурации для вашего сайта

```
sudo nano /etc/nginx/sites-available/your_project
```

### 2. Добавьте следующее содержимое в файл:

```
server {
 listen 80;
 server_name yourdomain.com www.yourdomain.com;

 location = /favicon.ico { access_log off; log_not_found off; }
 location /static/ {
 root /home/django_user/your_project_directory;
 }

 location /media/ {
 root /home/django_user/your_project_directory;
 }

 location / {
 include proxy_params;
 proxy_pass
http://unix:/home/django_user/your_project_directory/your_project.sock;
 }
}
```

#### Пояснение:

- `server_name` : Ваш домен или IP-адрес сервера.
- `root` : Путь к директории, где находятся статические и медиа-файлы.
- `proxy_pass` : Проксирование запросов к сокету Gunicorn.

### 3. Сохраните файл и выйдите из редактора.

### 4. Активируйте конфигурацию сайта

```
sudo ln -s /etc/nginx/sites-available/your_project /etc/nginx/sites-enabled
```

### 5. Проверьте конфигурацию Nginx на наличие ошибок

```
sudo nginx -t
```

### 6. Перезапустите Nginx



```
sudo systemctl restart nginx
```

## 7.2. Настройка брандмауэра (опционально)

Шаги:

1. Проверьте статус брандмауэра

```
sudo ufw status
```

2. Разрешите HTTP-трафик

```
sudo ufw allow 'Nginx Full'
```

3. Включите брандмауэр

```
sudo ufw enable
```

---

## Задача 8: Запуск и проверка работы приложения

### 8.1. Проверка работы приложения

Шаги:

1. Откройте веб-браузер и перейдите по адресу вашего сервера

```
http://yourdomain.com
```

Или по IP-адресу:

```
http://your_server_ip
```

2. Убедитесь, что ваше приложение работает корректно.

### 8.2. Проверка логов и устранение возможных ошибок

Шаги:

1. Просмотр логов Gunicorn

```
sudo journalctl -u gunicorn
```

## 2. Просмотр логов Nginx

```
sudo tail -f /var/log/nginx/error.log
sudo tail -f /var/log/nginx/access.log
```

## 3. Если возникают ошибки, проверьте конфигурацию и повторите настройки.

---

# Дополнительные рекомендации

- **Настройка SSL-сертификата**

- Для обеспечения безопасности рекомендуется настроить SSL-сертификат.
- Можно использовать бесплатный сертификат от Let's Encrypt с инструментом Certbot.

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

- **Автоматическое управление процессами**

- Убедитесь, что Gunicorn и Nginx автоматически запускаются при перезагрузке сервера.

- **Мониторинг и логирование**

- Настройте системы мониторинга и логирования для отслеживания состояния приложения.

- **Обновления и безопасность**

- Регулярно обновляйте систему и пакеты для обеспечения безопасности.
- Следите за обновлениями в вашем коде и зависимостях.

---

# Заключение

Поздравляем! Вы успешно:

- Развернули свой Django-проект на сервере в продакшн-среде.
- Настроили необходимые компоненты: базу данных, WSGI-сервер Gunicorn и веб-сервер Nginx.
- Обеспечили корректную работу приложения и его доступность для пользователей.

- Поняли общие принципы деплоя Django-приложений и можете применять их в будущем.
- 

**Успехов в разработке и деплое вашего приложения!**

---

## Вопросы для обсуждения

- **Почему важно разделять настройки для разработки и продакшн-среды в Django?**
  - Разделение настроек позволяет оптимизировать конфигурацию под конкретную среду. В разработке обычно включен параметр `DEBUG=True` для удобства отладки, а в продакшн-среде его необходимо отключить ( `DEBUG=False` ) для повышения безопасности. Также различаются параметры базы данных, разрешенные хосты ( `ALLOWED_HOSTS` ), обработка статических файлов и другие конфигурации, что помогает обеспечить безопасность и производительность приложения.
  - **Как Gunicorn и Nginx взаимодействуют при развертывании Django-приложения?**
  - Gunicorn является WSGI-сервером, который отвечает за выполнение Python-кода Django-приложения. Nginx выступает в роли обратного прокси-сервера, который принимает HTTP-запросы от пользователей и перенаправляет их на Gunicorn через сокет или TCP. Nginx также обрабатывает статические и медиа-файлы, обеспечивая более эффективное распределение нагрузки и повышая производительность приложения.
  - **Какие основные меры безопасности необходимо принять при деплое Django-проекта?**
  - При деплое Django-приложения важно настроить параметры безопасности, такие как `DEBUG=False` , правильно настроить `ALLOWED_HOSTS` , использовать секретный ключ из переменных окружения, настроить HTTPS с SSL-сертификатом для шифрования данных, ограничить доступ к админ-панели, использовать безопасные заголовки (например, `X-Frame-Options` , `X-Content-Type-Options` ), и регулярно обновлять зависимости проекта для устранения известных уязвимостей.
- 

## Полезные ресурсы

- [Официальная документация по развертыванию Django](#)
- [Руководство по деплою Django с Gunicorn и Nginx](#)

- [DigitalOcean: Как развернуть Django с Postgres, Nginx и Gunicorn](#)
- [Настройка SSL с Let's Encrypt](#)